

Evolução de Arquitetura: Servidor HTTP Simplificado sobre TCP/R-UDP e Resolução de Nomes Local

Gabriel Ramos Santos

Curso de Bacharelado em Sistemas de Informação
Universidade Federal do Piauí – UFPI
Campus Senador Helvídio Nunes de Barros
Redes de Computadores II
Matrícula: 20239042799

Resumo

This paper presents the evolution of a previous TCP and Reliable UDP file transfer system into a simplified web architecture. The new version implements a local DNS module over UDP and a simplified HTTP/1.1 web server capable of operating over native TCP and over the previously developed R-UDP reliability layer. The system was tested in Docker containers under different packet loss and delay scenarios using traffic control. The results compare DNS resolution time, HTTP transfer time, total loading time, throughput, retransmissions and success rate.

Resumo

Resumo. Este trabalho apresenta a evolução de um sistema anterior de transferência de arquivos via TCP e R-UDP para uma arquitetura Web simplificada. A nova versão implementa um módulo DNS local sobre UDP e um miniservidor HTTP/1.1 capaz de operar tanto sobre TCP nativo quanto sobre a camada de confiabilidade R-UDP desenvolvida anteriormente. O sistema foi testado em containers Docker sob diferentes cenários de perda de pacotes e atraso utilizando controle de tráfego. Os resultados comparam tempo de resolução DNS, tempo HTTP, tempo total de carregamento, throughput, retransmissões e taxa de sucesso.

1 Introdução

Esse projeto tem como objetivo evoluir o trabalho desenvolvido anteriormente, no qual foi implementada uma transferência de arquivos utilizando TCP nativo e UDP confiável. Nesta nova etapa, o sistema foi transformado em uma arquitetura Web simplificada, composta por cliente Web, servidor Web e servidor DNS local.

Diferentemente da avaliação anterior, em que o cliente enviava um arquivo diretamente ao servidor, nesta avaliação o cliente realiza uma consulta DNS antes de iniciar a comunicação HTTP. Após resolver o nome do servidor Web, o cliente envia uma requisição HTTP GET e recebe uma página HTML ou um arquivo estático. A comunicação HTTP foi implementada em dois modos: sobre TCP nativo e sobre a camada R-UDP.

2 Arquitetura da Solução

A arquitetura implementada possui três containers principais:

- **cliente_web**: responsável por consultar o DNS e realizar requisições HTTP via TCP ou R-UDP;
- **servidor_dns**: responsável por resolver nomes locais utilizando um arquivo `hosts.txt`;
- **servidor_web**: responsável por responder requisições HTTP/1.1 simplificadas.

O fluxo geral da aplicação ocorre da seguinte forma:

1. o cliente envia uma consulta DNS via UDP para o servidor DNS;
2. o servidor DNS consulta o arquivo `hosts.txt` e retorna o IP do servidor Web;
3. o cliente utiliza o IP recebido para iniciar a comunicação HTTP;
4. o servidor Web processa a requisição GET e responde com cabeçalhos HTTP e corpo do arquivo solicitado.

3 Módulo DNS Local

O módulo DNS local foi implementado sobre UDP. A consulta utiliza um formato simplificado contendo os campos ID e NAME. A resposta contém os campos ID, NAME e IP. O arquivo `hosts.txt` foi utilizado como tabela local de resolução de nomes.

Exemplo de entrada no arquivo `hosts.txt`:

```
www.meusite.local 172.30.0.10
```

Assim, antes de qualquer requisição HTTP, o cliente consulta o domínio `www.meusite.local` e recebe como resposta o IP `172.30.0.10`, correspondente ao container do servidor Web.

Como o DNS foi implementado de maneira simplificada sobre UDP, o cliente DNS também utiliza timeout na aplicação. Isso é necessário porque o UDP nativo não garante entrega, confirmação ou retransmissão automática. Caso a resposta não chegue dentro do tempo limite, o cliente identifica falha na resolução.

4 Miniservidor HTTP/1.1

O miniservidor HTTP implementado processa requisições do tipo GET e responde com cabeçalhos HTTP básicos. Os principais cabeçalhos utilizados foram:

- HTTP/1.1 200 OK;
- HTTP/1.1 404 Not Found;
- Content-Type;
- Content-Length;
- X-Custom-Auth;
- Connection: close.

O campo X-Custom-Auth foi mantido da avaliação anterior e contém o hash SHA-256 da concatenação da matrícula com o nome do aluno:

20239042799Gabriel Ramos Santos

O valor gerado e utilizado nas comunicações foi:

b16de223936c74916a7b2d34505b5c418e44545cfc0e66926abf27feefea363d

4.1 HTTP sobre TCP

No modo TCP, o cliente primeiro realiza a consulta DNS, depois abre uma conexão TCP com o servidor Web e envia uma requisição HTTP GET. O servidor responde com os cabeçalhos HTTP e o conteúdo do arquivo solicitado.

4.2 HTTP sobre R-UDP

No modo R-UDP, a requisição e a resposta HTTP são transportadas utilizando a camada de confiabilidade implementada sobre UDP. Essa camada utiliza:

- número de sequência;
- checksum MD5 por bloco;
- ACKs;
- timeout;
- retransmissão;
- marcador de fim de transmissão.

Dessa forma, a aplicação HTTP é transportada sobre o protocolo R-UDP, reaproveitando a camada criada na avaliação anterior.

5 Ambiente de Testes

Os testes foram realizados em ambiente Docker, com containers separados para cliente, servidor Web e servidor DNS. O comando `tc qdisc` foi utilizado para simular atraso e perda de pacotes no container do servidor Web.

```
>> docker compose up --build -d
=> => exporting config sha256:7bcd891abfacb8f93e05cd878345df2c93bba1419ad41dd1cf27032a15d59581
=> => exporting attestation manifest sha256:1a807b1320b362d0e98d5b4c016a1e702e5b44ac3b38faaa7ddd7df3966ab8eb
=> => exporting manifest list sha256:11be1df10eedbe7aabf845c5debfb685baf4ad43a494aca7bc1907a0e33a2d34
=> => naming to docker.io/library/trabalho02-redes-cliente_web:latest
=> => unpacking to docker.io/library/trabalho02-redes-cliente_web:latest
=> [servidor_dns] exporting to image
=> => exporting layers
=> => exporting manifest sha256:7f8173ab7c466839418c6ece38fee27b360ce3585ec7b54791d5e69ba38e69e0
=> => exporting config sha256:7bcd891abfacb8f93e05cd878345df2c93bba1419ad41dd1cf27032a15d59581
=> => exporting attestation manifest sha256:dbccff55ee507429dfda696185b45df7d19c2ae1e8ad6833e5252f81af593d2
=> => exporting manifest list sha256:7d66db79ea0482d7fa6d67dff05e4e455e4251012f533925443120b1b5d392b1
=> => naming to docker.io/library/trabalho02-redes-servidor_dns:latest
=> => unpacking to docker.io/library/trabalho02-redes-servidor_dns:latest
=> [servidor_web] exporting to image
=> => exporting layers
=> => exporting manifest sha256:7f8173ab7c466839418c6ece38fee27b360ce3585ec7b54791d5e69ba38e69e0
=> => exporting config sha256:7bcd891abfacb8f93e05cd878345df2c93bba1419ad41dd1cf27032a15d59581
=> => exporting attestation manifest sha256:c710c9ba4adea9b73a1231225b229d7f9f9c1a89b1213ae692224a0074be5b43
=> => exporting manifest list sha256:513c58a8daa169f98e2c33361ac885904f3e5d925a8fd611c9606fc4f20ced57
=> => naming to docker.io/library/trabalho02-redes-servidor_web:latest
=> => unpacking to docker.io/library/trabalho02-redes-servidor_web:latest
=> [cliente_web] resolving provenance for metadata file
=> [servidor_web] resolving provenance for metadata file
=> [servidor_dns] resolving provenance for metadata file
[+] Running 7/7
✓ servidor_web Built
✓ cliente_web Built
✓ servidor_dns Built
✓ Network trabalho02-redes_rede_trabalho Created
✓ Container servidor_web Started
✓ Container servidor_dns Started
✓ Container cliente_web Started
```

Figura 1: Containers Docker utilizados na arquitetura da aplicação.

Foram utilizados três cenários de rede:

Tabela 1: Cenários de teste		
Cenário	Delay	Perda de pacotes
A	10 ms	0%
B	50 ms	5%
C	100 ms	10%

```
root@72993b5c02ba:/app# tc qdisc add dev eth0 root netem delay 10ms loss 0%
root@72993b5c02ba:/app# python cliente http tcp.py
```

Figura 2: Configuração do Cenário A com 10 ms de delay e 0% de perda.

```
root@72993b5c02ba:/app# tc qdisc del dev eth0 root
root@72993b5c02ba:/app# tc qdisc add dev eth0 root netem delay 50ms loss 5%
```

Figura 3: Configuração do Cenário B com 50 ms de delay e 5% de perda.

```
root@72993b5c02ba:/app# tc qdisc del dev eth0 root
root@72993b5c02ba:/app# tc qdisc add dev eth0 root netem delay 100ms loss 10%
```

Figura 4: Configuração do Cenário C com 100 ms de delay e 10% de perda.

Foram utilizados três tamanhos de arquivo:

- 100 KB;
- 1 MB;
- 5 MB.

A escolha por 5 MB no lugar de 10 MB foi feita para reduzir o tempo total dos testes, mantendo a variação entre arquivos pequenos, médios e maiores.

6 Coleta de Dados

Para cada protocolo, cenário e tamanho de arquivo, foram executadas 10 repetições. Os resultados foram armazenados em arquivos CSV contendo:

- cenário;
- caminho do arquivo;
- tempo de resolução DNS;
- tempo HTTP;
- tempo total DNS + HTTP;
- tamanho da resposta;
- throughput;
- número de retransmissões no R-UDP;
- indicador de sucesso no R-UDP.

Os arquivos CSV gerados foram:

- `http_tcp_A.csv`;
- `http_tcp_B.csv`;
- `http_tcp_C.csv`;
- `http_rudp_A.csv`;
- `http_rudp_B.csv`;
- `http_rudp_C.csv`.

7 Resultados e Gráficos

Os gráficos foram gerados em Python utilizando Pandas e Matplotlib. A análise considera o impacto do DNS, do HTTP e das condições de rede sobre o desempenho do TCP e do R-UDP.

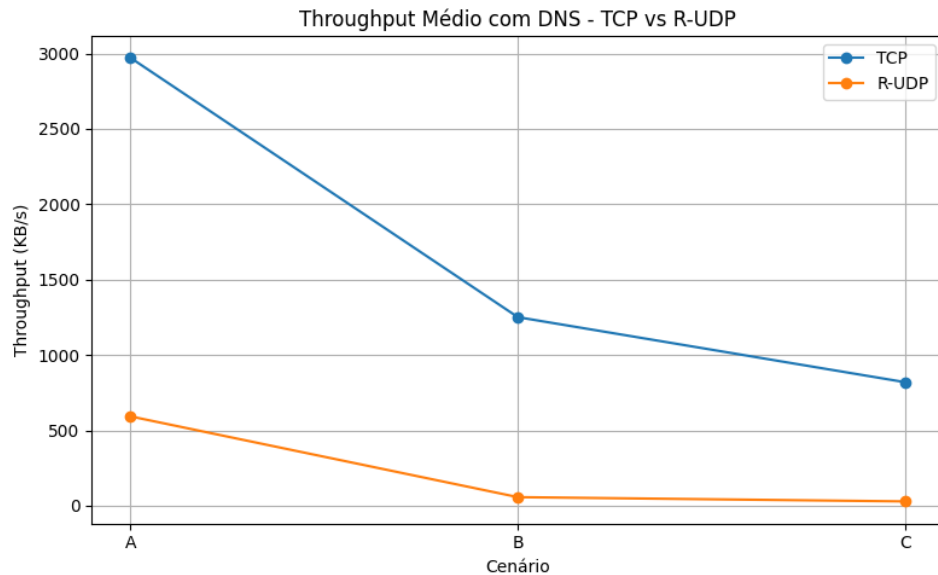


Figura 5: Throughput médio com DNS comparando TCP e R-UDP nos três cenários.

O gráfico de throughput mostra a diferença de desempenho entre TCP e R-UDP. O TCP tende a apresentar maior taxa de transferência, enquanto o R-UDP sofre maior impacto com perda de pacotes e atraso, principalmente por utilizar uma estratégia baseada em confirmação e retransmissão.

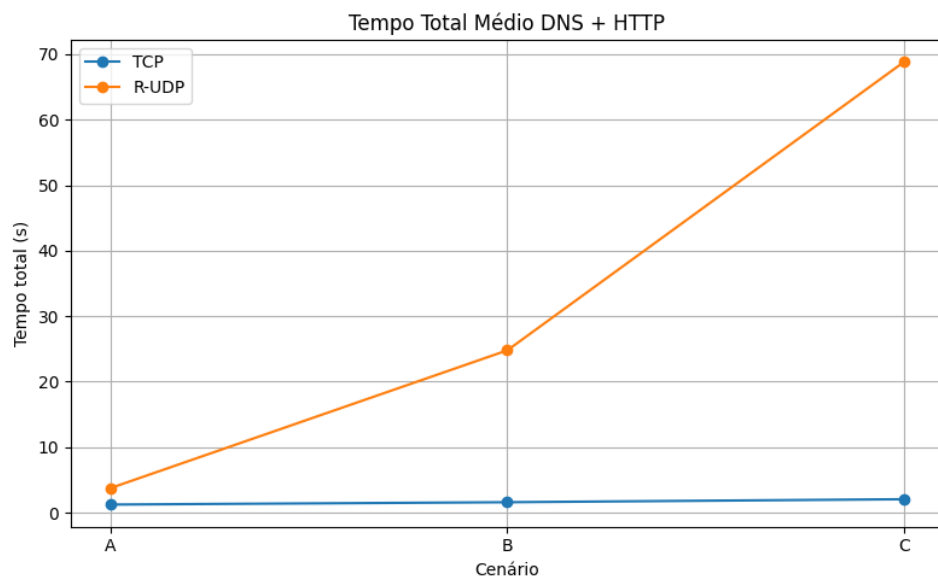


Figura 6: Tempo total médio de carregamento considerando DNS + HTTP.

O tempo total inclui tanto a etapa de resolução DNS quanto a transferência HTTP. Em cenários com maior delay e perda, o tempo total tende a aumentar, principalmente no R-UDP.

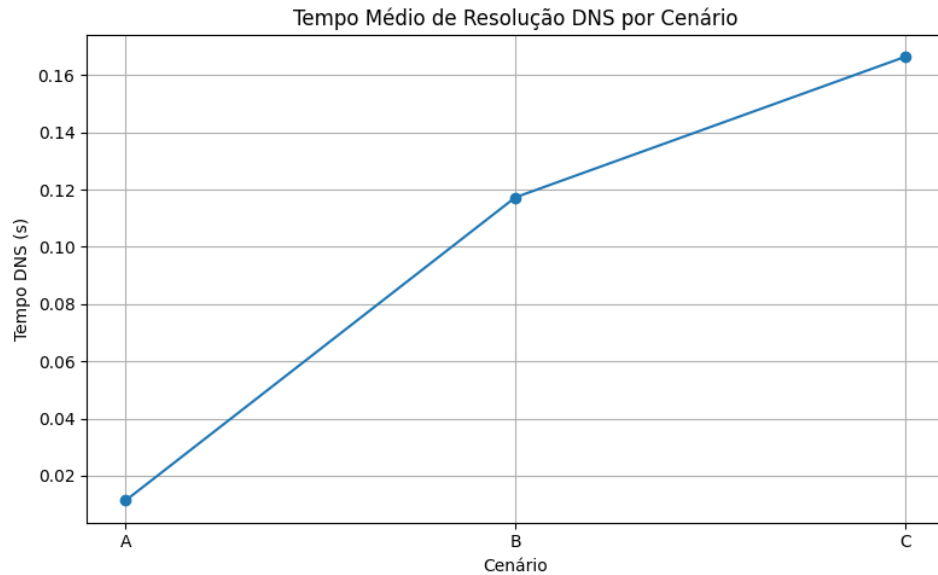


Figura 7: Tempo médio de resolução DNS nos três cenários.

O tempo DNS foi medido separadamente para verificar o impacto da perda de pacotes no protocolo UDP nativo. Como o DNS simplificado não possui confiabilidade na camada de transporte, perdas podem gerar timeout na aplicação.

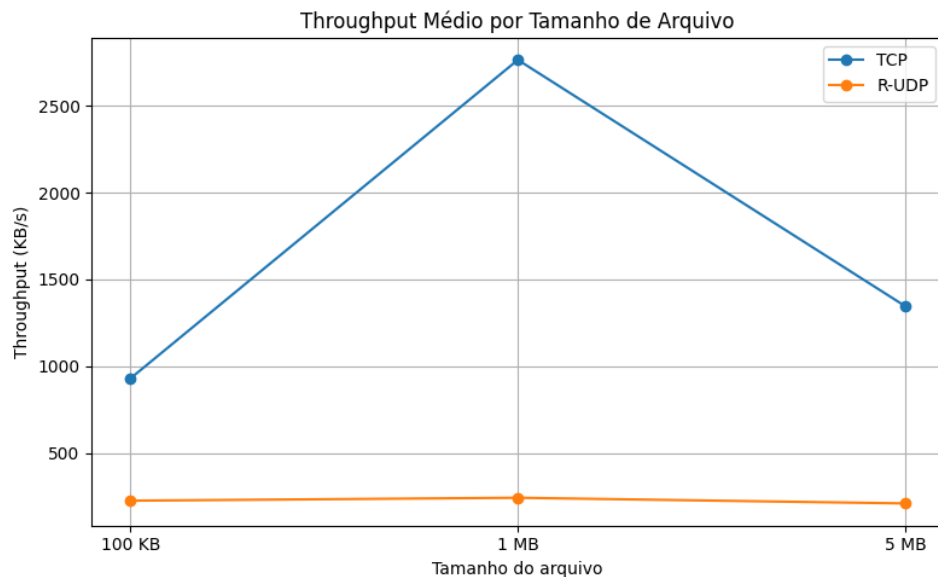


Figura 8: Throughput médio por tamanho de arquivo.

A comparação por tamanho de arquivo permite observar que arquivos maiores tendem a evidenciar melhor as diferenças entre TCP e R-UDP, pois exigem maior

quantidade de pacotes e, conseqüentemente, ficam mais sujeitos aos efeitos de atraso e perda.

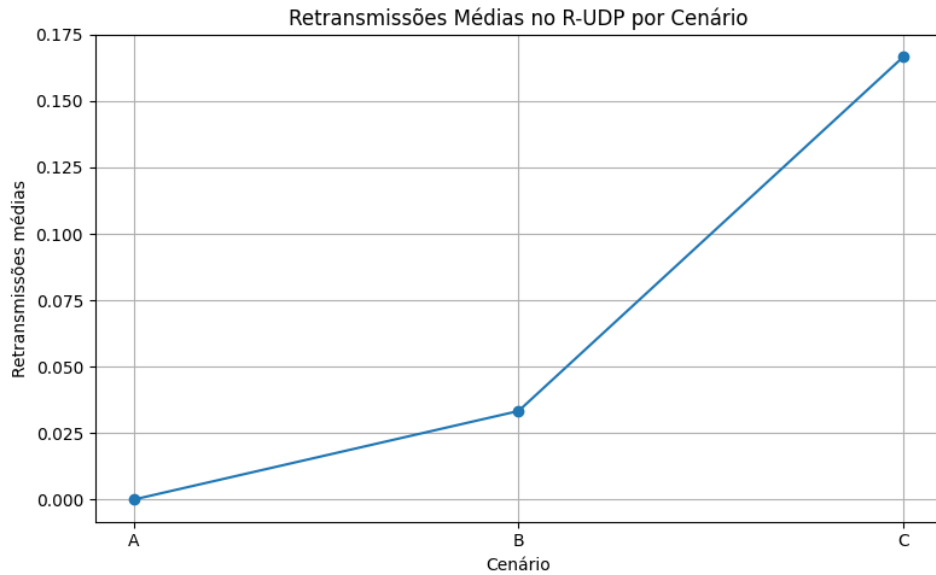


Figura 9: Retransmissões médias no R-UDP por cenário.

As retransmissões no R-UDP aumentam conforme a perda de pacotes cresce. Isso é esperado, pois o protocolo implementado precisa reenviar pacotes quando não recebe confirmação dentro do tempo limite.

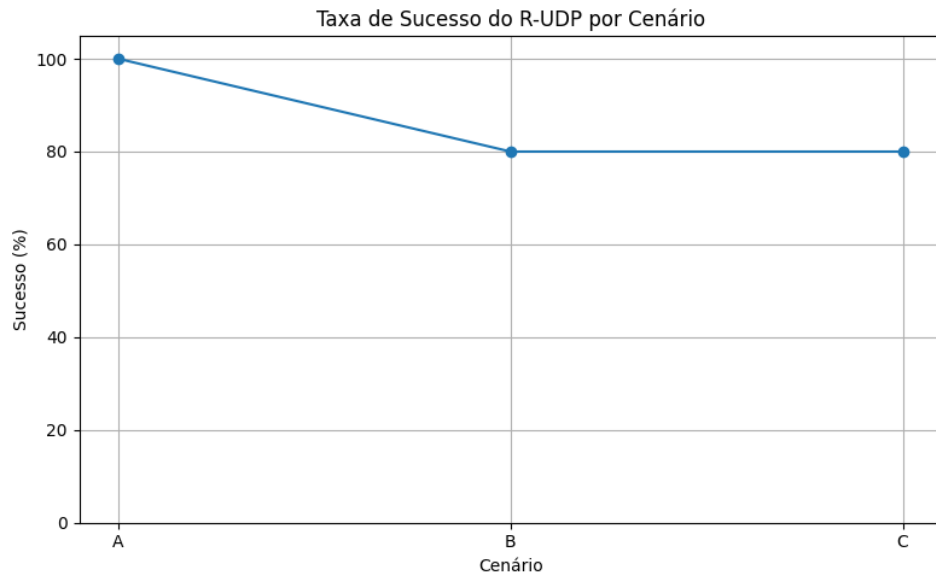


Figura 10: Taxa de sucesso do R-UDP por cenário.

A taxa de sucesso indica quantas requisições foram concluídas corretamente no R-UDP. Em cenários com maior perda, podem ocorrer falhas ou finalizações por timeout, dependendo do comportamento da rede simulada.

8 Validação com Wireshark/TCPDump

A validação de rede foi realizada com capturas .pcap geradas por TCPDump e analisadas no Wireshark. O objetivo foi comprovar a sequência lógica da comunicação: primeiro a resolução DNS e depois a comunicação HTTP via TCP ou R-UDP.

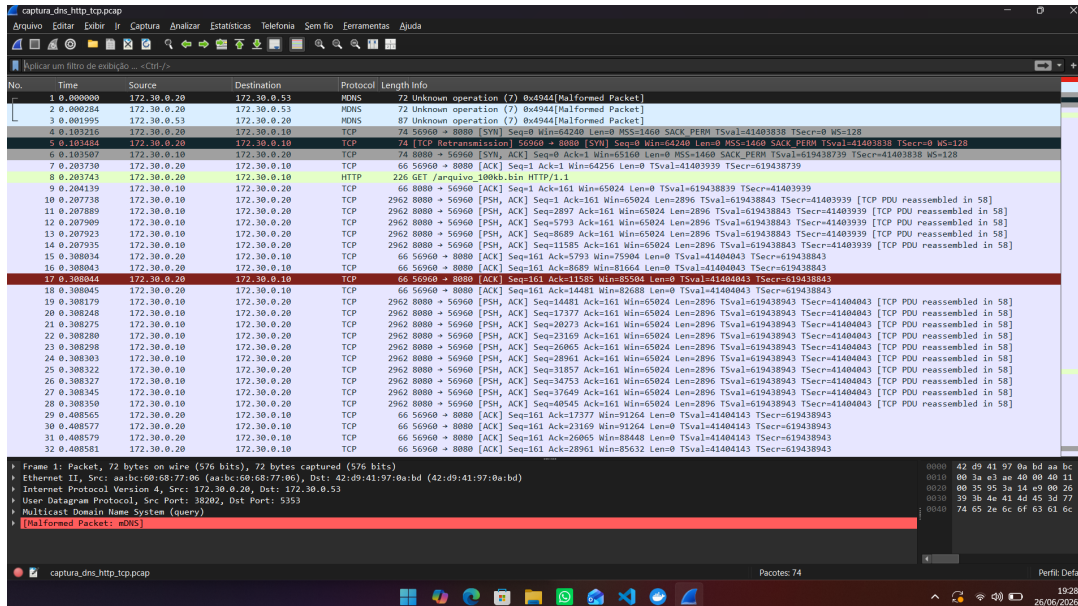


Figura 11: Captura mostrando a sequência DNS seguida da comunicação HTTP via TCP.

Na captura TCP, é possível observar o cliente 172.30.0.20 consultando o servidor DNS 172.30.0.53. Em seguida, ocorre o início da comunicação com o servidor Web 172.30.0.10. Isso comprova que a resolução de nomes ocorre antes da requisição HTTP.

Como o DNS foi implementado em formato simplificado sobre UDP, o Wireshark pode identificar os pacotes como mDNS malformados. Isso ocorre porque não foi implementado o cabeçalho DNS real completo, mas sim o formato simplificado exigido na aplicação.

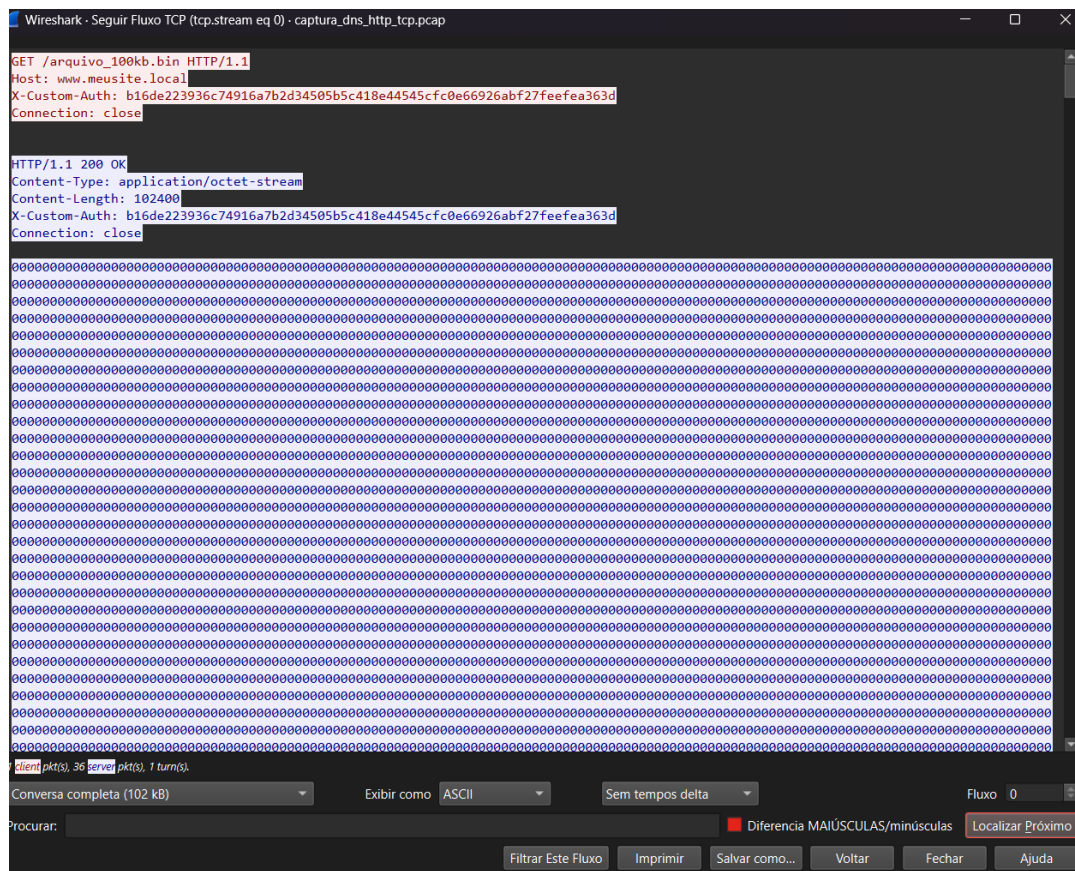


Figura 12: Fluxo TCP contendo a requisição GET, a resposta HTTP/1.1 200 OK e o campo X-Custom-Auth.

O fluxo TCP mostra a requisição:

```
GET /arquivo_100kb.bin HTTP/1.1
Host: www.meusite.local
X-Custom-Auth: ...
```

e a resposta:

```
HTTP/1.1 200 OK
Content-Type: application/octet-stream
Content-Length: 102400
X-Custom-Auth: ...
```

No.	Time	Source	Destination	Protocol	Length	Info
6	39.121893	172.30.0.20	172.30.0.10	UDP	244	35021 → 8081 Len=202
7	39.122170	172.30.0.10	172.30.0.20	UDP	46	8081 → 35021 Len=4
8	39.222557	172.30.0.20	172.30.0.10	UDP	45	35021 → 8081 Len=5
14	39.222777	172.30.0.10	172.30.0.20	UDP	876	8081 → 35021 Len=8234
15	39.328000	172.30.0.20	172.30.0.10	UDP	46	35021 → 8081 Len=4
21	39.328009	172.30.0.10	172.30.0.20	UDP	876	8081 → 35021 Len=8234
22	39.429500	172.30.0.20	172.30.0.10	UDP	46	35021 → 8081 Len=4
28	39.430406	172.30.0.10	172.30.0.20	UDP	876	8081 → 35021 Len=8234
29	39.531152	172.30.0.20	172.30.0.10	UDP	46	35021 → 8081 Len=4
35	39.532004	172.30.0.10	172.30.0.20	UDP	876	8081 → 35021 Len=8234
41	40.534292	172.30.0.10	172.30.0.20	UDP	876	8081 → 35021 Len=8234
42	40.635005	172.30.0.20	172.30.0.10	UDP	46	35021 → 8081 Len=4
48	40.635338	172.30.0.10	172.30.0.20	UDP	876	8081 → 35021 Len=8234
49	40.734175	172.30.0.20	172.30.0.10	UDP	46	35021 → 8081 Len=4
55	40.734924	172.30.0.10	172.30.0.20	UDP	876	8081 → 35021 Len=8234
56	40.835732	172.30.0.20	172.30.0.10	UDP	46	35021 → 8081 Len=4
62	40.836369	172.30.0.10	172.30.0.20	UDP	876	8081 → 35021 Len=8234
63	40.937281	172.30.0.20	172.30.0.10	UDP	46	35021 → 8081 Len=4
69	40.937820	172.30.0.10	172.30.0.20	UDP	876	8081 → 35021 Len=8234
75	41.939961	172.30.0.10	172.30.0.20	UDP	876	8081 → 35021 Len=8234
76	42.040928	172.30.0.20	172.30.0.10	UDP	46	35021 → 8081 Len=4
82	42.041583	172.30.0.10	172.30.0.20	UDP	876	8081 → 35021 Len=8234
83	42.142083	172.30.0.20	172.30.0.10	UDP	46	35021 → 8081 Len=4
89	42.143220	172.30.0.10	172.30.0.20	UDP	876	8081 → 35021 Len=8234
98	42.244205	172.30.0.20	172.30.0.10	UDP	46	35021 → 8081 Len=4
96	42.244998	172.30.0.10	172.30.0.20	UDP	876	8081 → 35021 Len=8234
97	42.345809	172.30.0.20	172.30.0.10	UDP	47	35021 → 8081 Len=5
103	42.346470	172.30.0.10	172.30.0.20	UDP	876	8081 → 35021 Len=8234
104	42.447592	172.30.0.20	172.30.0.10	UDP	47	35021 → 8081 Len=5
107	42.448208	172.30.0.10	172.30.0.20	UDP	1403	8081 → 35021 Len=4321
108	42.549184	172.30.0.20	172.30.0.10	UDP	47	35021 → 8081 Len=5
109	42.549650	172.30.0.10	172.30.0.20	UDP	45	8081 → 35021 Len=3

Frame 7: Packet, 46 bytes on wire (368 bits), 46 bytes captured (368 bits)
 Ethernet II, Src: 68:8d:2e:35:78:f0 (68:8d:2e:35:78:f0), Dst: aa:bc:60:68:77:06 (aa:bc:60:68:77:06)
 Internet Protocol Version 4, Src: 172.30.0.10, Dst: 172.30.0.20
 User Datagram Protocol, Src Port: 8081, Dst Port: 35021
 Data (4 bytes)

0000 aa bc 60 68 77 06 c6 8d 2e 35 78 f0
 0010 00 20 c2 44 4b 0a 4b 11 74 00 00
 0020 00 14 1f 91 88 cd 00 0c 50 00 00

Figura 13: Captura do tráfego HTTP sobre R-UDP utilizando pacotes UDP entre cliente e servidor Web.

No caso do R-UDP, o tráfego ocorre sobre UDP na porta definida para o servidor HTTP R-UDP. A captura mostra os pacotes UDP enviados entre cliente e servidor Web, representando a troca confiável implementada na aplicação.

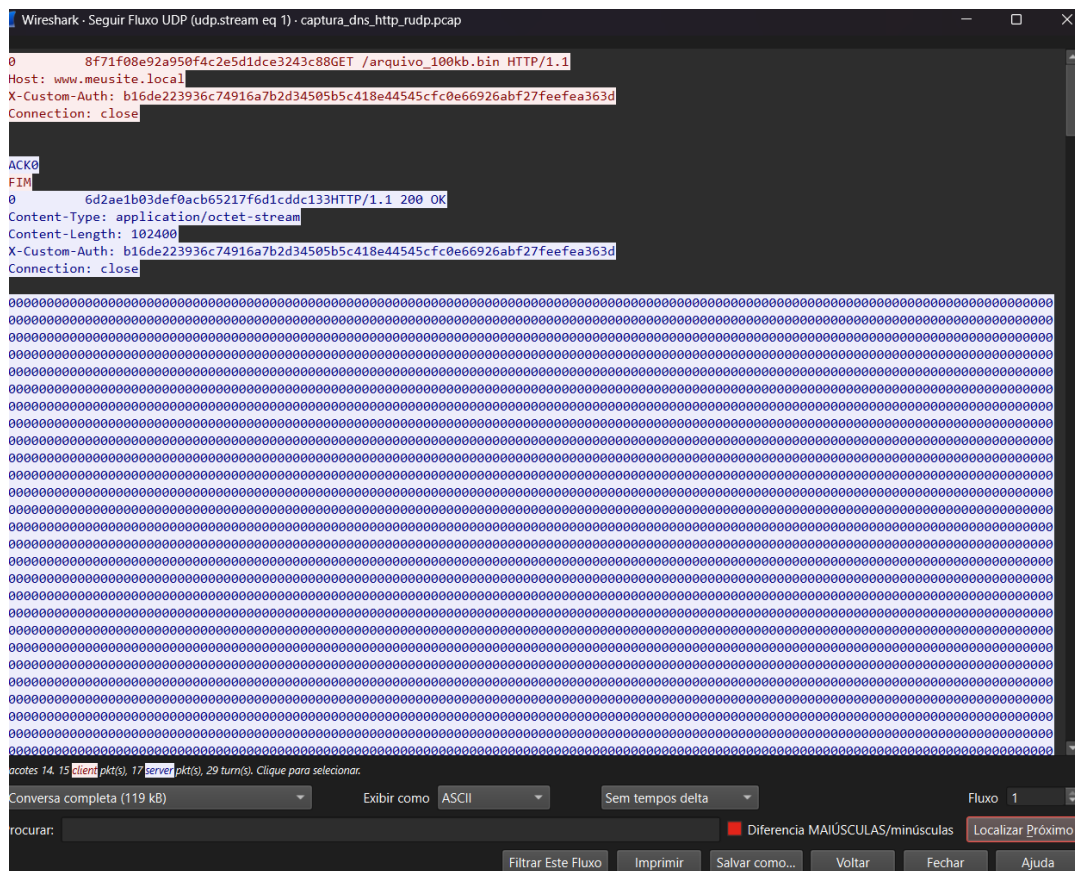


Figura 14: Fluxo UDP contendo a requisição HTTP transportada sobre R-UDP, ACKs, marcador FIM e resposta HTTP.

O fluxo UDP mostra que a requisição HTTP também foi transportada sobre a camada R-UDP. É possível observar a requisição GET, os ACKs, o marcador de fim de transmissão e a resposta HTTP contendo os cabeçalhos obrigatórios.

9 Overhead dos Cabeçalhos HTTP

Na avaliação anterior, o protocolo de aplicação era mais simples e personalizado, contendo apenas informações mínimas para transferência e validação dos dados. Nesta avaliação, foi adicionado o cabeçalho HTTP/1.1, o que aumentou o overhead da comunicação.

Um exemplo de requisição HTTP utilizada foi:

```
GET /arquivo_100kb.bin HTTP/1.1
Host: www.meusite.local
X-Custom-Auth: b16de223936c74916a7b2d34505b5c418e44545cfc0e66926abf27feefea363d
Connection: close
```

A resposta inclui:

```
HTTP/1.1 200 OK
```

```
Content-Type: application/octet-stream
Content-Length: 102400
X-Custom-Auth: b16de223936c74916a7b2d34505b5c418e44545cfc0e66926abf27feefea363d
Connection: close
```

Esse overhead é mais perceptível em arquivos pequenos, como o arquivo de 100 KB. Em arquivos maiores, como 1 MB e 5 MB, o impacto relativo dos cabeçalhos diminui, pois o corpo da resposta representa a maior parte dos bytes transferidos.

10 Respostas às Perguntas Obrigatórias

10.1 1. Como a perda de pacotes afetou o tempo de resolução DNS em comparação com o download HTTP via R-UDP/TCP? O cliente DNS precisou implementar timeout?

A perda de pacotes afeta o DNS de forma direta, pois o módulo DNS utiliza UDP nativo, sem retransmissão automática na camada de transporte. Por esse motivo, o cliente DNS precisou implementar timeout na aplicação para evitar que a consulta ficasse bloqueada indefinidamente esperando uma resposta.

Em comparação, o download HTTP via TCP é mais robusto porque o TCP já possui mecanismos nativos de confiabilidade, controle de fluxo, confirmação e retransmissão. Já no caso do HTTP via R-UDP, a confiabilidade foi implementada na aplicação utilizando ACKs, checksum, timeout e retransmissões. Assim, o DNS fica mais vulnerável à perda de pacotes do que o HTTP via TCP, enquanto o HTTP via R-UDP depende da eficiência da camada de confiabilidade implementada.

10.2 2. Qual foi o impacto visual e métrico do overhead dos cabeçalhos HTTP em comparação ao protocolo customizado anterior?

O overhead dos cabeçalhos HTTP aumentou a quantidade de dados transmitidos em relação ao protocolo textual customizado da avaliação anterior. Agora, cada requisição e resposta contém linhas HTTP como `GET`, `Host`, `Content-Type`, `Content-Length`, `Connection` e `X-Custom-Auth`.

Visualmente, no Wireshark, esse overhead aparece de forma clara no fluxo TCP e UDP, pois é possível observar os cabeçalhos HTTP antes do corpo do arquivo. Métricamente, o impacto é mais relevante nos arquivos menores, como 100 KB, pois o tamanho dos cabeçalhos representa uma parcela maior do total transmitido. Para arquivos maiores, como 1 MB e 5 MB, o impacto relativo diminui, porque o corpo do arquivo domina o volume total da transferência.

10.3 3. Ao inspecionar o Wireshark, o fluxo seguiu a ordem esperada da arquitetura da Internet?

Sim. As capturas mostraram a sequência esperada. Primeiro ocorre a consulta DNS entre o cliente e o servidor DNS. Em seguida, após a resposta contendo o IP do servidor Web, o cliente inicia a comunicação HTTP.

No caso do TCP, a captura mostra a consulta DNS, a resposta DNS, o início do handshake TCP e depois a requisição GET. No caso do R-UDP, a captura mostra a consulta DNS e, em seguida, a primeira transmissão UDP contendo a requisição HTTP encapsulada na camada R-UDP. Portanto, a ordem observada foi compatível com a arquitetura esperada: resolução de nomes antes da transferência HTTP.

11 Conclusão

Este trabalho evoluiu a aplicação da avaliação anterior para uma arquitetura Web simplificada. Foi implementado um servidor DNS local sobre UDP, um miniservidor HTTP/1.1 com suporte a GET, resposta 200 OK, erro 404, cabeçalhos básicos e o campo X-Custom-Auth. Além disso, o sistema foi capaz de operar tanto sobre TCP nativo quanto sobre a camada R-UDP desenvolvida anteriormente.

Os testes mostraram que o TCP apresentou maior estabilidade e desempenho nos cenários de perda e atraso, enquanto o R-UDP sofreu maior impacto devido à necessidade de confirmações e retransmissões na aplicação. O DNS, por utilizar UDP nativo, exigiu timeout no cliente para lidar com possíveis perdas. As capturas no Wireshark confirmaram a sequência lógica da comunicação, com DNS ocorrendo antes do HTTP, e demonstraram a presença dos cabeçalhos HTTP e do campo X-Custom-Auth.

Como melhoria futura, seria possível implementar uma camada R-UDP mais eficiente, utilizando janelas deslizantes como Go-Back-N ou Selective Repeat, além de um DNS simplificado com retransmissão automática na aplicação.